

From Fragmentation to Cohesion: An LLM-Based Iterative Approach to Ontology and Knowledge Graph Refinement

Alexandru Popa

Department of Computer Science

University of Bucharest

National Institute for Research and Development

Bucharest, Romania

alexandru.popa@fmi.unibuc.ro

Tiberiu-Iulian Sîrbu

Department of Computer Science

University of Bucharest

Bucharest, Romania

sirbu.tiberiu.iulian@gmail.com

Abstract—Knowledge graphs and their respective ontologies often suffer from disconnected subgraphs, redundant classes, and inconsistent relationships when automatically generated from unstructured data. While recent studies involving Large Language Models (LLMs) are capable of generating knowledge graphs, they lack the capability to offer a well-connected and stable graph. In this paper, a new pipeline is proposed that repeatedly refines the ontology by identifying disconnected components of the knowledge graph and employs the LLM to find the connections between the largest connected component and all the smaller disconnected ones. Moreover, the pipeline discussed in this study aims to offer a stable ontology at the end that will create a consistent graph each time is used as input in a LLM based knowledge graph construction.

Index Terms—Knowledge Graphs, Large Language Models, Graph Connectivity Analysis, Automated Ontology Generation.

I. INTRODUCTION

Recent advancements in Large Language Models (LLMs) offer the opportunity of using this type of technology to obtain answers from a vast amount of data. The most common way of achieving answers from data used nowadays is known as Retrieval Augmented Generation (RAG) [11]. This method takes a number of documents and splits them into contexts before transforming each context into embeddings and storing them in vector stores. Each time a user asks a question, it will use the query’s embeddings to perform a semantic similarity and extract the closest context to the actual query.

Although RAG effectively extracts textual information from documents, it lacks the ability to capture contextual relationships between entities. Knowledge Graphs (KGs) address this limitation by explicitly connecting entities, even those not directly stated in the text. Constructing KGs requires advanced knowledge of data and an ontology rules consisting of classes and properties guiding entity extraction and relationship identification by the LLM.

KGs underpin GraphRAG, a novel method designed to overcome traditional RAG limitations by better interpreting complex semantic relationships. While recent research has significantly increased interest in GraphRAG [2], [4], [10], [16],

this study specifically focuses on improving the quality and coherence of KG construction, rather than directly enhancing KG-based retrieval mechanisms.

A. Related Work

Traditional KG building pipelines frequently yield disconnected subgraphs, largely due to incomplete ontology or conflicting domain knowledge. In parallel, LLMs have recently demonstrated significant capabilities in extracting, aligning, and validating knowledge by taking advantage of the extensive semantic context encoded within their parameters [12]. However, LLM-based KG construction has far often focused on partial challenges such as zero-shot triple extraction [3], ontology alignment [13], or canonicalization steps [15], without providing an end-to-end mechanism to iteratively consolidate disconnected components into a single coherent graph.

Over the past two to three years, researchers have integrated LLMs into various portions of the KG lifecycle. Such an example is iText2KG, which incrementally builds a KG from text to reduce the need for post hoc merging [9]. Graphusion is another framework, which fuses partial subgraphs via a global “fusion” prompt, boosting overall connectivity [14]. Meanwhile, studies highlighting how class membership relations in KGs can be evaluated against LLM-sourced definitions, uncovers subtle errors in large knowledge repositories [1]. Similar researches show that open-world hierarchical knowledge from LLMs can improve link prediction [6], or that LLM can be used as a “graph judge” which repeatedly checks extracted facts for internal consistency [5].

B. Original Contribution

Despite other pipelines for ontology creation with LLM exists [8], a comprehensive pipeline that explicitly identifies disconnected subgraphs, prioritizes their integration (ordering nodes by degree), and iteratively revises the ontology until the entire graph converges has been largely absent. Moreover, there is limited discussion on using graph similarity across

repeated instantiations, an approach that can help detect residual inconsistencies and imply whether the ontology is stable enough for deployment. The present article aims to fill these gaps by proposing a novel multi-stage algorithm that:

- 1) Generates an initial ontology from the source documents using an LLM.
- 2) Builds a KG, identifying all connected components.
- 3) Systematically merges smaller disconnected subgraphs into the largest component by prompting the LLM to suggest to refine classes or relations.
- 4) Repeats ontology refinement until changes drop below a defined threshold.
- 5) Rebuilds the KG twice with the final ontology and measures graph similarity to verify stability.

By adopting this iterative, connectivity-driven perspective, this method aims to offer a clear stopping criterion when the ontology demonstrates consistent, reproducible connectivity. The remainder of this paper is organized as follows: Section II describes the theoretical considerations used and defined the pipeline for ontology creation. Before concluding the current study in Section IV, the Section III aims to showcase the effectiveness of the proposed pipeline with results from an experiment conducted on a document linked to chemistry domain.

II. THEORETICAL CONSIDERATIONS

In this section, the theoretical foundations are outlined along with the assumptions underlying the proposed pipeline for iterative ontology construction and KG refinement using LLMs. The present approach systematically addresses two critical challenges in automated knowledge graph creation from text: semantic coherence (ensuring accurate concept relations) and structural coherence (ensuring a fully connected graph).

In tackling the semantic convergence, the assumption is that iterative prompting of an LLM will produce progressively refined ontology definitions. Over iterations, the ontology changes become progressively smaller, eventually converging towards a stable semantic representation. By prioritizing connections between nodes based on their degree within graph components, the pipeline aims to rapidly increase the connectivity of the KG. Nodes with higher degrees are more semantically central, making them effective anchors for integration. The iterative nature ensures the largest component acts as the primary reference, progressively absorbing smaller disconnected subgraphs.

After ontology refinement converges, reconstructing the KG multiple times with the same ontology provides a measure of the ontology’s stability. High similarity between these repeated graphs indicates that the ontology reliably captures the domain’s semantic structure, while lower similarity suggests ongoing instability or ambiguity. A Jaccard-like measure is employed on the set of edges. This choice reflects a combinatorial perspective: two KGs are considered the “same” if they share most of their edges. In network science, other measures (e.g., graph edit distance, subgraph isomorphism tests) could

Algorithm 1 Iterative Ontology & KG Refinement Pipeline

```

1: Input: Documents  $D$ , similarity threshold  $\tau$ , convergence threshold  $\epsilon$ 
2: Output: Final Ontology  $O_{\text{final}}$ , Final KG  $G_{\text{final}}$ 
3: Generate initial ontology  $O$  from documents  $D$  via LLM
4: Extract RDF triples from documents  $D$  based on ontology  $O$ 
5: Construct initial KG  $G$  from extracted triples
6: repeat
7:   Identify connected components in KG  $G$ 
8:   Assign largest connected component as main graph  $C_{\text{main}}$ 
9:   Mark smaller components as  $C_{\text{small}}$ 
10:  for each component  $C_j$  in  $C_{\text{small}}$  do
11:    Sort nodes in  $C_j$  by descending node degree
12:    Sort nodes in  $C_{\text{main}}$  by descending node degree
13:    Use LLM to find semantic relationships between top-degree nodes of  $C_j$  and  $C_{\text{main}}$ 
14:    if LLM suggests valid connection then
15:      Update ontology  $O$  accordingly
16:    end if
17:  end for
18:  Rebuild KG  $G$  using updated ontology  $O$ 
19:  Measure ontology changes  $\Delta O$ 
20: until  $\Delta O < \epsilon$ 
21: Construct graphs  $G_A$  and  $G_B$  independently from final ontology  $O$ 
22: Compute similarity  $\text{sim}(G_A, G_B)$ 
23: if  $\text{sim}(G_A, G_B) < \tau$  then
24:   Repeat ontology refinement (go to Step 6)
25: end if
26: return  $O_{\text{final}}, G_{\text{final}}$ 

```

apply, but Jaccard distance is used in this study to balance simplicity with descriptive power.

The complete pipeline depicted in Algorithm 1 provides a structured theoretical foundation for automated ontology and KG refinement. It integrates graph theory principles (connectivity, node degree) and LLM-driven semantic inference (relationship identification, ontology update proposals) into a practical iterative framework.

The algorithm starts by generating an initial ontology from input documents using an LLM. This ontology is then used to extract Resource Description Framework (RDF) triples and construct the initial knowledge graph.

The core of the algorithm is an iterative loop that identifies disconnected components in the KG. The largest component is treated as the main graph, while the smaller components are processed to find potential semantic links. Nodes from both the smaller and main components are sorted by degree, and the LLM is prompted to find valid relationships between them. When a connection is identified, the ontology is updated accordingly.

This refinement cycle continues until changes to the ontology become negligible. Once the final ontology is stable, two

separate KGs are constructed from it, and their similarity is measured. If the graphs are sufficiently similar, the final ontology and KG are returned; otherwise, refinement continues.

The pipeline outputs an ontology that not only accurately reflects the domain knowledge contained in the input documents but also reliably generates a stable, well-connected knowledge graph.

III. EXPERIMENTS AND RESULTS

In this section, the experimental results of the iterative ontology refinement pipeline and knowledge graph creation are presented. For experiments a structured, domain-specific input article was chosen [7]. This article was selected due to its clear domain specific terminology, hierarchical relationships, and structured content, making it an ideal candidate for knowledge extraction and ontology refinement.

RDF triples serve as the fundamental building blocks for semantic data representation. Each RDF triple consists of three components: a subject, predicate, and object, which together define relationships between entities clearly and explicitly. In the current study, RDF triples were used to capture semantic relationships extracted from the input document.

The pipeline implementation used several key technologies and libraries. The OpenAI API provided a connection to an LLM, specifically employing the GPT-4O mini model. This model was chosen for its state-of-the-art generative capabilities, enabling accurate identification and extraction of domain-specific concepts and relationships directly from the text. Furthermore, we utilized PyVis, a Python library specialized in interactive network visualization, to clearly represent and analyze the structure of the resulting RDF graphs. Additionally, NetworkX, a powerful graph analysis library, was used for computing graph connectivity, identifying connected components, and sorting nodes by their degree, thus guiding the iterative refinement process.

To effectively manage the input article text, a text chunking technique was applied. The original document was split into smaller, overlapping chunks, facilitating manageable inputs for the GPT-4O mini model. Given the nature of pricing when using an LLM, all iterative pipeline creation as the one defined in present article, may be expensive, as the price increases with the number of documents and number of iterations used.

The threshold for ontology change was set to 0.1, while the threshold for final graph similarity was set to 0.7. Figure 1 depicts the initial knowledge graph generated using the ontology directly produced by GPT-4O mini from the input document. The graph clearly illustrates a disconnected structure, with multiple smaller components isolated from the central, larger subgraph. This fragmentation indicates gaps or missing semantic connections within the initial ontology, limiting the utility and completeness of the KG.

Through the iterative process of identification of high degree nodes, connectivity checks, and further semantic interrogation via LLM, the ontology was progressively updated to incorporate previously missing relationships. The iterative nature of this refinement allowed the model to propose meaningful

connections between the disconnected subgraphs and the main connected component, effectively enhancing both the semantic accuracy and connectivity of the ontology.

The effectiveness of our iterative pipeline is clearly demonstrated in Figure 2, which shows the final refined KG after multiple refinement cycles. This graph is characterized by a unified, coherent structure with a single dominant connected component. All previously isolated subgraphs are now fully integrated, showcasing significantly improved semantic connectivity and completeness.

Additionally, Table I presents a comparative analysis of the initial and final knowledge graphs generated before and after the ontology refinement process. The metrics clearly reflect the structural improvements brought by our iterative pipeline.

First, a reduction in the total number of nodes is observed, from 68 to 55. This decrease suggests that the refinement process effectively resolved redundancies and merged semantically equivalent entities. Conversely, the number of edges increased from 64 to 67, indicating that the refined ontology facilitated the discovery of additional relationships between entities. This improvement in edge density also contributes to greater graph connectivity and coherence.

A critical result is the reduction in the number of connected components from 5 in the initial KG to just 1 in the final version. This demonstrates that the pipeline successfully integrated previously isolated subgraphs into a unified, semantically connected structure. The size of the largest component grew from 38 nodes to encompass the entire graph (55 nodes), while the concept of a “smallest component” no longer applies in the refined graph, since it has become a single connected component.

Furthermore, the average node degree increased significantly from 1.88 to 2.44. This increase illustrates that, on average, each entity in the final KG is linked to more related concepts, contributing to better semantic richness and improved context availability for downstream tasks such as question answering or inference.

Finally, the RDF triple count closely mirrors the edge count, reinforcing that each new semantic link corresponds directly to enriched information content in the graph.

TABLE I
COMPARISON OF INITIAL AND FINAL KNOWLEDGE GRAPHS

Metric	Initial KG	Final KG
Total Nodes	68	55
Total Edges	64	67
Number of Connected Components	5	1
Largest Component Size (nodes)	38	55
Smallest Component Size (nodes)	2	55
Average Node Degree	1.88	2.44
RDF Triple Count	64	67

Another important detail is the obtained similarity between the final graphs created from the final ontology. This similarity yielded 0.746, while this value may reflect a difference in the graph creation using the same ontology, further refinement of prompting with additional rules to guide the LLM in RDF triples construction, may result in a more stable approach.

Overall, the results demonstrate that iterative ontology refinement using advanced language models can substantially enhance KG connectivity and semantic coherence, providing robust frameworks suitable for structured domain knowledge extraction and representation.

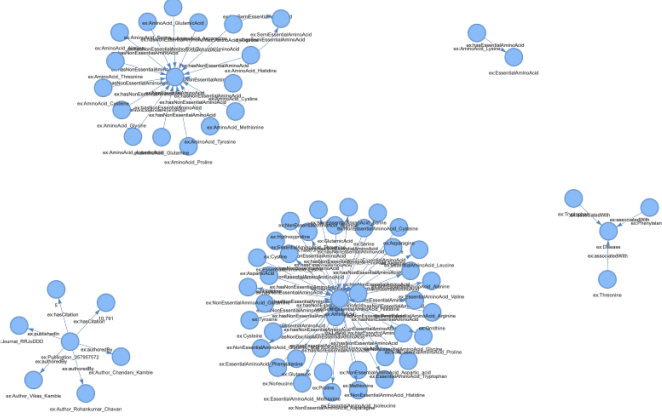


Fig. 1. Initial KG



Fig. 2. Final KG

IV. CONCLUSIONS

In this study, a novel iterative ontology refinement pipeline was introduced and was designed to improve semantic coherence and connectivity of knowledge graphs derived from structured domain-specific documents. The key contribution of this paper lies in systematically addressing disconnected components within knowledge graphs through repeated ontology refinements guided by language models. The pipeline employs RDF triples for precise representation of semantic relationships, enabling explicit and structured knowledge extraction. This methodology demonstrated its effectiveness through the transformation from an initially fragmented knowledge graph

to a fully connected, semantically enriched graph. These results highlight the potential of iterative ontology refinement to substantially improve knowledge graph quality and reliability, paving the way for enhanced performance in downstream applications such as knowledge discovery, semantic search, and automated reasoning. Future research directions include exploring pipeline scalability, extending the methodology to broader domains, improve consistency in graph creation based on ontology, and integrating automated reasoning capabilities to further validate the refined ontologies.

REFERENCES

- [1] B. P. Allen and P. T. Groth. Evaluating class membership relations in knowledge graphs using large language models. In *European Semantic Web Conference*, pages 14–24. Springer, 2024.
- [2] M. Barry, G. Caillaut, P. Halftermeyer, R. Qader, M. Mouayad, D. Carliolario, F. Le Deit, and J. Gesnoui. Graphrag: Leveraging graph-based efficiency to minimize hallucinations in llm-driven rag for finance data. In *31st International conference on Computational Linguistics Workshop Knowledge Graph & GenAI*, 2025.
- [3] S. Carta, A. Giuliani, L. Piano, A. S. Podda, L. Pompianu, and S. G. Tiddia. Iterative zero-shot llm prompting for knowledge graph construction. *arXiv preprint arXiv:2307.01128*, 2023.
- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [5] H. Huang, C. Chen, C. He, Y. Li, J. Jiang, and W. Zhang. Can llms be good graph judger for knowledge graph construction? *arXiv preprint arXiv:2411.17388*, 2024.
- [6] P. Jiang, L. Cao, C. D. Xiao, P. Bhatia, J. Sun, and J. Han. Kg-fit: Knowledge graph fine-tuning upon open-world knowledge. *Advances in Neural Information Processing Systems*, 37:136220–136258, 2024.
- [7] C. Kamble, R. Chavan, and V. Kamble. A review on amino acids. research & reviews: A journal of drug design & discovery. 2021; 8 (3): 19–27p. *A Review on Amino Acids Kamble et al. STM Journals*, page 2, 2021.
- [8] V. K. Kommineni, B. König-Ries, and S. Samuel. From human experts to machines: An llm supported approach to ontology and knowledge graph construction. *arXiv preprint arXiv:2403.08345*, 2024.
- [9] Y. Lairgi, L. Moncla, R. Cazabet, K. Benabdeslem, and P. Cléau. itext2kg: Incremental knowledge graphs construction using large language models. In *International Conference on Web Information Systems Engineering*, pages 214–229. Springer, 2024.
- [10] M.-C. Lee, Q. Zhu, C. Mavromatis, Z. Han, S. Adeshina, V. N. Ioannidis, H. Rangwala, and C. Faloutsos. Hybgrag: Hybrid retrieval-augmented generation on textual and relational knowledge bases. *arXiv preprint arXiv:2412.16311*, 2024.
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [12] J. Z. Pan, S. Razniewski, J.-C. Kalo, S. Singhania, J. Chen, S. Dietze, H. Jabeen, J. Omeliyanenko, W. Zhang, M. Lissandrini, et al. Large language models and knowledge graphs: Opportunities and challenges. *arXiv preprint arXiv:2308.06374*, 2023.
- [13] M. Taboada, D. Martinez, M. Arideh, and R. Mosquera. Ontology matching with large language models and prioritized depth-first search. *arXiv preprint arXiv:2501.11441*, 2025.
- [14] R. Yang, B. Yang, A. Feng, S. Ouyang, M. Blum, T. She, Y. Jiang, F. Lecue, J. Lu, and I. Li. Graphusion: a rag framework for knowledge graph construction with a global perspective. *arXiv preprint arXiv:2410.17600*, 2024.
- [15] B. Zhang and H. Soh. Extract, define, canonicalize: An llm-based framework for knowledge graph construction. *arXiv preprint arXiv:2404.03868*, 2024.
- [16] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu. Knowledge graph-guided retrieval augmented generation. *arXiv preprint arXiv:2502.06864*, 2025.